

Cómo desarrollé Atlas Spaces desde cero

Diario técnico del proceso — Prueba Técnica Full Stack, Quantum Infinity Technologies

1. Entendiendo el problema antes de escribir una sola línea

Antes de tocar el teclado, leí el PDF completo dos veces. La prueba no pedía “una app bonita”, pedía algo muy concreto: un sistema de reservas de coworking (Atlas Spaces) con dos roles (admin y operador), reglas de negocio estrictas sobre solapamiento de horarios, un dashboard con datos reales calculados en el backend, exportación a CSV, y todo corriendo en Docker. Seis tickets obligatorios, ninguno opcional.

Lo primero que hice no fue código: fue una lista con los sustantivos y verbos clave del documento — “usuario”, “espacio”, “reserva”, “rol”, “estado”, “solapamiento”, “auditoría”. Esa lista terminó siendo, casi literalmente, el modelo de datos.

Lo que ya sabía: cómo estructurar una API REST con Express, cómo modelar datos con Mongoose, y cómo montar un frontend en React con rutas protegidas.

Lo que no sabía tan bien: JWT y bcrypt los había tocado antes, pero no a fondo — había hecho algún login sencillo en algún curso o proyecto pequeño, sin entrar en detalles como expiración de tokens, verificación de rol en el middleware o por qué el hash de la contraseña no se puede comparar con ===. Para este reto tuve que reforzar esa parte en serio: repasar cómo firma y verifica tokens jsonwebtoken, y entender bien qué hace bcrypt al comparar (bcrypt.compare) en vez de simplemente “encriptar y ya”.

También me costó manejar correctamente una zona horaria (America/Bogotá) cuando MongoDB guarda todo en UTC, y hacer que “reservas consecutivas” (una que empieza exactamente cuando termina otra) no chocara con la regla de solapamiento. Esta última fue, sin duda, la parte que más problemas me dio de todo el reto: en el primer intento escribí la condición de solapamiento con `<=` y `>=` en vez de `<` y `>`, lo que rechazaba reservas consecutivas por error. Lo detecté al escribir la prueba automatizada para ese caso específico — el test fallaba una y otra vez hasta que entendí que la comparación debía ser estrictamente **`startA < endB && endA > startB`**. Fue el típico error de “un signo mal puesto” que casi se me pasa por alto si no hubiera escrito esa prueba.

2. Decisión de arquitectura y stack

El documento ya definía el stack obligatorio (React + Router + Tailwind, Node + Express, MongoDB, Docker), así que ahí no había mucho que decidir. Las decisiones reales fueron:

- **JavaScript en vez de TypeScript.** TypeScript sumaba puntos, pero preferí no arriesgar tiempo en configurar tipos para un proyecto de fin de semana con alcance grande. Fue una decisión consciente de priorización, no de comodidad: prefería seis tickets completos en JS que cuatro tickets en TS.

- **Estructura de carpetas por responsabilidad** (controllers/, models/, routes/, middleware/, utils/) en vez de por “feature”. La elegí porque el equipo que revisa el código necesita encontrar rápido “dónde está la regla de negocio” sin adivinar, y esta estructura es la más estándar en proyectos Express medianos.
- **Git flow simplificado:** una rama por ticket (feature/ticket1-auth, feature/ticket2-espacios-reservas, etc.) que se mergea a develop. Esto no lo pedía el documento explícitamente, pero sabía que “un solo commit final” se penalizaba, así que decidí forzar el hábito de trabajar con un historial que se pueda seguir ticket por ticket. Vale la pena ser preciso sobre esto: los tickets se desarrollaron en secuencia, no en paralelo (soy un solo desarrollador, no un equipo) — las ramas por ticket fueron una forma de organizar y presentar el avance de manera legible, no un flujo con features realmente divergentes que se cruzaran entre sí.

3. Backend, archivo por archivo

3.1 backend/src/config/db.js

Aquí solo vive la conexión a Mongoose. La separé de app.js a propósito: si el día de mañana cambio de MongoDB local a Atlas, o necesito reconectar con retry, este es el único archivo que toco. Es una decisión pequeña pero es la diferencia entre código mantenible y código que hay que buscar por todo el proyecto.

3.2 backend/src/models/User.js, Space.js, Reservation.js

Estos tres modelos salieron casi directo de la tabla “Modelo mínimo de información” del PDF. Lo único que añadí por mi cuenta —porque el documento no lo detallaba— fue:

- **passwordHash** en vez de guardar la contraseña en texto plano, cifrada con bcrypt.
- **Timestamps automáticos** de Mongoose (createdAt/updatedAt) para cubrir el requisito de “fechas de auditoría” sin reinventar la rueda.
- Un array **BLOCKING_STATUSES = ['pending', 'confirmed']** exportado desde el modelo de Reserva, porque esa lista se repite en varias partes del código (reglas de negocio, dashboard) y prefería tener una sola fuente de verdad en vez de escribir los mismos strings en cinco archivos distintos. (Nota: mantuve los valores del enum en inglés — 'pending', 'confirmed', 'cancelled', 'completed' — por consistencia con el resto de nombres de campos y constantes del código, que también están en inglés; solo las etiquetas visibles en el frontend, como “Pendiente” o “Confirmado”, están en español.)

Cosa que no sabía y tuve que investigar: cómo Mongoose maneja fechas internamente. Todo se guarda como Date en UTC de forma nativa en Mongo, así que decidí que toda la lógica de negocio trabaja en UTC y solo convierto a hora Bogotá en el momento de mostrar datos o validar horarios de apertura/cierre del espacio (que sí están pensados en hora local). Esa fue una decisión de diseño que

documenté para mí mismo en comentarios dentro de `dateUtils.js`, porque sabía que si no la explicaba, en tres semanas ni yo mismo iba a recordar por qué convertía en un punto y no en otro.

3.3 backend/src/utils/dateUtils.js

Funciones puras: `nowUTC()`, `utcDateToBogotaHHmm()`, `utcDateToBogotaISOString()`. Las separé de la lógica de negocio a propósito para poder probarlas de forma aislada (`dateUtils.unit.test.js`) sin tener que levantar una base de datos falsa solo para comprobar que una hora se convierte bien.

3.4 backend/src/utils/reservationRules.js

El archivo que más tiempo me tomó pensar, no escribir. Contiene una función por regla (`validateDateOrder`, `validateNotInPast`, `validateAttendeesCapacity`, `validateWithinSpaceSchedule`, `validateSpaceActive`, `assertNoOverlap`) y una función orquestadora `validateReservationRules` que las llama todas en orden. La dividí así, en vez de un único bloque gigante de `ifs`, porque:

- Cada regla se puede probar por separado.
- Cuando el negocio pida cambiar una regla (por ejemplo, permitir reservas en el pasado para un caso administrativo especial), solo se toca una función, no un monolito.

Aquí también definí una clase `BusinessRuleError` con su propio `statusCode`, para que el controller no tenga que decidir “esto es un 409 o un 422” — la regla que falla ya sabe qué código HTTP le corresponde. Esto lo aprendí revisando cómo estructuran errores personalizados otros proyectos Express; no era algo que ya supiera hacer bien antes de este reto.

3.5 backend/src/middleware/auth.js

`requireAuth` valida el JWT y adjunta `req.user`; `requireRole(...roles)` es una función que devuelve un middleware, para poder escribir `requireRole('admin')` de forma legible en las rutas. Cuidé especialmente que los mensajes de error no revelaran si el correo existe o no (“credenciales inválidas” genérico), porque el documento pedía explícitamente no exponer información sensible del servidor o del usuario.

3.6 backend/src/middleware/errorHandler.js y httpLogger.js

El manejador de errores centralizado evita repetir `try/catch` con `res.status().json()` en cada controller — junto con `express-async-handler`, cualquier error lanzado en un controller `async` cae aquí y se traduce a una respuesta HTTP coherente. `httpLogger.js` usa `pino-http` para registrar cada request; esto no era obligatorio, lo agregué después de completar los seis tickets, como mejora adicional, porque el documento mencionaba explícitamente “logs” como algo valorado.

3.7 Autenticación completa (Ticket 1) y refresh tokens

La primera versión solo tenía un access token JWT de larga duración. Después, como mejora, implementé `RefreshToken.js` (un modelo separado que guarda el hash del refresh token, no el token

en texto plano) y cookies httpOnly para el refresh. Esto lo aprendí investigando durante el reto: no sabía de antes por qué se recomienda no guardar el refresh token tal cual en base de datos — la razón es que si alguien compromete la base de datos, no debería poder usar los tokens directamente; por eso se guarda un hash, igual que con las contraseñas.

3.8 Espacios y Reservas (Ticket 2)

spaceController.js y reservationController.js separan claramente permisos por rol: el operador puede leer espacios pero el middleware requireRole('admin') bloquea crear/editar/desactivar, devolviendo 403 aunque intente pegarle directo al endpoint (no solo ocultando el botón en el frontend, que es un error común y el documento lo señala explícitamente como algo negativo: “validaciones únicamente visuales que puedan omitirse consumiendo la API”).

Para el listado de reservas escribí una función compartida buildReservationFilter(query) que arma el filtro de Mongo a partir de status, spaceId, from, to y search. La reutilicé también en la exportación CSV (ver 3.9) para no duplicar la lógica de filtros en dos lugares — si mañana cambia un filtro, solo se cambia en un sitio.

3.9 Exportación CSV (Ticket 5)

exportController.js reutiliza buildReservationFilter y buildSort del controller de reservas, sin paginación, para exportar exactamente los mismos resultados que el usuario está viendo filtrados. Añadí un BOM UTF-8 ( FEFF) al inicio del archivo porque, sin   , Excel en Windows muestra mal los acentos — esto lo tuve que buscar, no lo sabía de antes.

3.10 Dashboard y anal tica (Ticket 3)

analyticsController.js usa el pipeline de agregaci n de MongoDB (\$match, \$group) para calcular total de reservas, confirmadas, tasa de cancelaci n y el espacio con m s reservas confirmadas, todo dentro del rango de fechas seleccionado. Decid  que todo el c culo vive en el backend, ni una suma se hace en React, porque el documento lo ped a expl citamente y porque adem s evita que tarjetas y gr ficos muestren n meros inconsistentes entre s  (otro punto que el documento marcaba como cr tico).

Honestidad aqu : las agregaciones de Mongo no las domino de memoria; tuve que revisar la sintaxis de \$group con \$sum condicional (\$cond) para la tasa de cancelaci n, porque no la recordaba exacta.

3.11 Pruebas automatizadas (Ticket 4)

Us  Jest + Supertest en el backend. Eleg  cubrir exactamente los cuatro casos m nimos que ped a el documento (autenticaci n v lida, reserva rechazada por solapamiento, reserva aceptada por ser consecutiva, validaci n de capacidad/horario/fechas) y agreg  dos casos m s que a m  me parec an importantes: que al editar una reserva no se compare contra s  misma, y las pruebas unitarias de dateUtils. Prefer  pocos tests bien pensados que cubran los casos l mite reales, en vez de muchos tests triviales que no aportan confianza real.

Resultado final verificado: corrí ambas suites de extremo a extremo, con MongoDB real (mongodb-memory-server descarga y levanta un motor de MongoDB genuino, temporal y aislado). Backend: 3 suites, **24 de 24 pruebas pasando** (auth.test.js, reservations.test.js, dateUtils.unit.test.js). Frontend: 4 archivos, **19 de 19 pruebas pasando** con Vitest (StatusBadge, Pagination, ConfirmDialog, AuthContext).

3.12 Dockerización (Ticket 6)

docker-compose.yml levanta Mongo, backend y frontend con healthchecks (el backend no arranca “sano” hasta que Mongo responde). Usé un volumen nombrado (mongo-data) para que los datos sobrevivan a un docker compose down sin -v. El README explica también cómo correr todo sin Docker, tal como pedía el documento, porque depender de Docker para depurar es más lento que correr Node directo en la máquina.

Verificado: ejecuté docker compose up --build de extremo a extremo con Docker Desktop — los tres contenedores (frontend, backend, MongoDB) levantan correctamente, el seed carga los datos iniciales, y la aplicación funciona igual que en modo local sin Docker.

4. Frontend, archivo por archivo

4.1 api/client.js y api/resources.js

Un cliente Axios centralizado con el interceptor de refresh token, y un archivo separado con funciones por recurso (getSpaces, createReservation, etc.) para que las páginas nunca llamen a axios directamente. Si cambia la URL base o la forma de autenticar, se toca un solo archivo.

4.2 context/AuthContext.jsx y hooks/useAuth.js

El contexto guarda el usuario actual y expone login/logout. Lo separé del hook (useAuth) siguiendo el patrón estándar de React: el contexto tiene la lógica, el hook es solo el punto de acceso, para no repetir useContext(AuthContext) en cada componente.

4.3 components/ProtectedRoute.jsx

Protege rutas según si hay sesión y, opcionalmente, según el rol requerido. Redirige a login si no hay token válido — el equivalente en frontend de lo que requireAuth/requireRole hacen en backend. Ninguno de los dos reemplaza al otro: el frontend oculta, el backend prohíbe de verdad.

4.4 pages/ReservationsPage.jsx

La página más grande del frontend (351 líneas). Junta filtros, paginación, tabla, modal de creación/edición y botón de exportar CSV. Podría haberla dividido en más subcomponentes, y es honestamente la parte que más volvería a tocar si tuviera más tiempo — está en el límite de lo que considero “un componente que todavía se entiende de un vistazo”.

4.5 components/ReservationFormModal.jsx y SpaceFormModal.jsx

Formularios controlados con validación en frontend (mensajes inmediatos) que replican pero no reemplazan la validación del backend — el backend es la única fuente de verdad real, el frontend solo mejora la experiencia para no esperar un roundtrip por cada error obvio.

4.6 components/UiStates.jsx

Componentes reutilizables para los cuatro estados que el documento pedía explícitamente: cargando, error, sin datos, y confirmación antes de acciones sensibles (ConfirmDialog.jsx). Los centralicé para no repetir el mismo `if (loading) return <Spinner />` en cada página.

4.7 hooks/useModalA11y.js y accesibilidad

Esto lo agregué después de tener los seis tickets funcionando, como mejora: manejo de foco al abrir/cerrar modales, cierre con tecla Escape, roles ARIA. No sabía bien de antes cómo implementar “trampa de foco” (focus trap) correctamente; lo investigué específicamente para esta parte porque quería que fuera una mejora real y no solo cosmética.

4.8 Pruebas de frontend

Con Vitest: pruebas de componentes pequeños (Pagination, StatusBadge, ConfirmDialog) y del AuthContext. Las elegí porque son unidades pequeñas y aisladas — no intenté simular toda la aplicación, sino verificar las piezas que, si se rompen, rompen silenciosamente la experiencia del usuario.

4.9 El sidebar responsive y un bug de clases CSS conflictivas

El diseño original no contemplaba pantallas angostas: el menú lateral tenía un ancho fijo que, en un celular, dejaba casi sin espacio al contenido. Lo convertí en un drawer (menú deslizable) oculto por defecto en móvil, con un botón de hamburguesa y un overlay oscuro para cerrarlo. Al implementarlo cometí un error que no detecté hasta verificarlo visualmente: la clase del componente aplicaba al mismo tiempo `-translate-x-full` (la posición “oculto”, siempre presente) y `translate-x-0` (la posición “visible”, agregada solo cuando el menú está abierto) sobre el mismo elemento — dos clases de Tailwind controlando la misma propiedad CSS a la vez. Funcionaba, pero por casualidad del orden interno en que Tailwind genera las reglas en el CSS final, no porque el código estuviera bien escrito. Lo corregí reemplazando el patrón por un operador ternario que aplica una clase u otra, nunca las dos a la vez, para que dejara de depender de un detalle de implementación de la herramienta.

4.10 Verificar el responsive sin las herramientas ideales a mano

No tenía un navegador headless disponible en el entorno donde desarrollé esta parte con asistencia de IA (Chromium requería snap, que no estaba disponible; y las descargas de Puppeteer/Chrome for Testing estaban bloqueadas por la configuración de red del entorno). La alternativa fue usar wkhtmltoimage — un motor de renderizado basado en WebKit — para generar capturas reales a distintos anchos de pantalla, y además confirmar directamente en el CSS ya compilado que las clases

responsive de Tailwind existían y no se habían eliminado por accidente durante la optimización del build. Fue un paso más lento de lo que hubiera sido con Chrome DevTools, pero preferí esa vía a asumir sin evidencia que el resultado se veía como se esperaba.

4.11 Por qué no invertí más en diseño visual

El documento de la prueba dice explícitamente que “no se evalúa un diseño de alta fidelidad”, y que en cambio se mide jerarquía visual, legibilidad, consistencia, responsive, mensajes útiles y facilidad para completar las tareas principales. Tomé esa frase en serio como una señal de dónde no debía gastar el tiempo limitado de un proyecto de fin de semana con seis tickets obligatorios: no en pulir cada pantalla con microinteracciones, ilustraciones personalizadas o animaciones, sino en tener un sistema de diseño simple pero consistente — una paleta de color con significado real (verde para confirmado/disponible, ocre para pendiente, rojo para cancelado/peligro, no colores decorativos sin propósito), una jerarquía tipográfica clara, y los mismos componentes de estado (carga, error, vacío, confirmación) reutilizados en toda la aplicación en vez de reinventados por pantalla.

Invertir en un diseño más elaborado tenía además un costo de oportunidad concreto: cada hora en pulir estética era una hora menos para verificar que las reglas de negocio del Ticket 4 fueran correctas, que es lo que el propio documento trata con más detalle y con más casos límite explícitos. Y hay un riesgo adicional que solo entendí bien al final del proyecto: un diseño más ambicioso, sin la posibilidad real de verificarlo en distintos tamaños de pantalla de forma confiable, corre el riesgo de verse peor que uno modesto pero probado — como me pasó con el propio comportamiento responsive (ver 4.9 y 4.10), que tuve que verificar con capturas reales en vez de asumir que el CSS generado se comportaba como esperaba. Preferí un diseño con menos ambición visual pero con más certeza de que funciona en la práctica, que uno más ambicioso pero sin verificar.

5. Uso de inteligencia artificial — con honestidad

Esta sección ya no es un borrador: refleja el IA.md real que terminé escribiendo y que sí quedó consistente con el proyecto.

En general, el uso fue mitad y mitad: para partes que no dominaba tanto (como el detalle fino de JWT/bcrypt, o las agregaciones de MongoDB) usé la IA para generar una primera versión y entenderla revisándola; para la lógica de negocio que sí tenía clara en la cabeza (las reglas de reservas, la estructura de carpetas) escribí yo primero y usé la IA más para revisar y detectar casos que se me pudieran estar escapando.

- **Herramienta usada:** Claude (Anthropic), como asistente principal para diseñar la arquitectura, generar el código base de backend y frontend, y redactar la documentación.
- **Tareas para las que se usó:** el modelo de datos completo a partir del PDF de requerimientos, la implementación de los seis tickets (auth, espacios/reservas, dashboard, reglas de negocio, CSV, Docker), y la redacción de las pruebas automatizadas y de esta misma documentación.

- **Cómo se dio contexto:** se le entregó a la IA el documento completo de la prueba —los seis tickets, el modelo mínimo de datos, las condiciones técnicas y los criterios de evaluación— como base de todo el desarrollo, para que cada decisión se validara contra un requerimiento explícito y no contra un diseño genérico.

Tres respuestas de IA que se descartaron o corrigieron, y por qué:

- **Zona horaria:** la IA sugirió primero usar una librería completa de zonas horarias (tipo luxon) con base de datos IANA. Se descartó porque Colombia no tiene horario de verano — su offset (UTC-5) es fijo todo el año, así que esa dependencia era sobreingeniería. Se reemplazó por una conversión explícita y simple, documentada en el código.
- **Búsqueda de texto:** la IA propuso un índice \$text de MongoDB. Se descartó porque \$text solo hace coincidencia por palabra completa, no por subcadena (buscar “andr” no encontraría “Andrés”). Se reemplazó por una búsqueda con regex, escapando caracteres especiales para evitar inyección.
- **mongodb-memory-server en las pruebas:** se detectó, fuera del entorno con IA, que el runner de tests necesita descargar un binario de Mongo por internet la primera vez. Como el sandbox de desarrollo tenía la red restringida, no se pudo correr la suite completa de extremo a extremo ahí — se documentó explícitamente en el README en vez de ocultarlo. Ya en mi propia máquina, con internet normal, la suite completa corrió sin problema (ver 3.11).

Qué se validó manualmente: la lógica de solapamiento de horarios se revisó línea por línea para confirmar que permite reservas consecutivas sin permitir solapamientos reales (el error típico de `<=` en vez de `<`). También se corrigió a mano un índice duplicado en el modelo User, y el orden de las rutas de Express — `/api/reservations/export` debía declararse antes que `/api/reservations/:id`, o Express interpretaba “export” como un `:id` y el endpoint de exportación nunca se ejecutaba.

Una decisión que NO se delegó a la IA: qué hacer con las reservas futuras de un espacio cuando se desactiva —el propio documento de la prueba señalaba esto como una “decisión no definida” a propósito—. Se decidió no cancelar automáticamente esas reservas, solo bloquear nuevas sobre el espacio y devolver una advertencia con el conteo de reservas afectadas, para que el administrador decida. La razón: cancelar compromisos ya adquiridos con clientes reales es una acción destructiva que no debería ejecutarse sin confirmación humana explícita.

6. Lo que falta — y lo que casi queda mal documentado

Ser transparente aquí es literalmente parte de la evaluación, así que van dos tipos de honestidad: la del código, y la del propio proceso de escribir esta documentación.

Sobre el código, lo que de verdad sigue pendiente:

- `ReservationsPage.jsx` es grande. Con más tiempo, se dividiría en subcomponentes (`ReservationFilters`, `ReservationTable`, `ReservationActions`) para que cada uno se pruebe y se lea por separado.

- No hay CI configurado. Las pruebas corren localmente con un comando documentado (npm test en backend y frontend, ambos verificados pasando de extremo a extremo — ver 3.11), pero no en cada push. Identificado como mejora futura, no como parte del alcance mínimo.
- Cobertura de pruebas de frontend es parcial. Sí existen pruebas con Vitest (componentes puntuales y AuthContext), pero no cubren flujos completos de página de principio a fin.

Sobre Docker: en una primera etapa del desarrollo no pude validarlo de extremo a extremo porque el entorno donde trabajaba con asistencia de IA no tenía Docker disponible. Ya en mi propia máquina, con Docker Desktop, corrí docker compose up --build de extremo a extremo sin ajustes — los tres contenedores levantan correctamente (ver 3.12).

Sobre el proceso de documentar: al escribir el README por primera vez, subí por error un archivo de otro proyecto mío (un backend en Django para gestión de monitorías universitarias) en vez del README real de Atlas Spaces. Lo detecté a tiempo porque, antes de dar la entrega por terminada, contrasté el README contra el código real línea por línea — y ahí salió también un segundo problema más sutil: una primera versión corregida del README describía el proyecto como si no tuviera refresh tokens, pruebas de frontend ni documentación OpenAPI, cuando en realidad esas tres cosas ya estaban implementadas (se habían agregado como mejoras después de completar los seis tickets, y el README nunca se actualizó). La lección real de este reto no fue solo escribir las reglas de negocio correctamente — fue no dar por hecho que un documento está bien solo porque “se ve completo”; hay que releerlo contra el código tantas veces como se relee el código contra los requerimientos.

7. Cierre

Terminar este reto me deja más consciente de en qué punto estoy como desarrollador que si simplemente hubiera entregado una lista de funcionalidades marcadas como “listas”. Hay partes donde sentí que ya sé pararme solo: la estructura del backend, separar responsabilidades, pensar el modelo de datos antes de escribir código. Y hay partes donde se notó que todavía estoy construyendo esa base: JWT y bcrypt los había tocado por encima antes, no a fondo, y tuve que reforzarlos en serio durante estos días en vez de asumir que ya los sabía. La regla de solapamiento de horarios me costó más de lo que esperaba — un simple $\leq$ en vez de $<$ casi se me queda sin detectar, y solo lo atrapé porque escribí la prueba para ese caso específico. Eso me deja una lección clara: escribir el test del caso límite no es un paso extra al final, es lo que realmente revela si entendiste la regla o solo la copiaste.

No llego a este cierre pensando que ya domino todo lo que toqué aquí — llego pensando que ahora sé con más precisión qué es lo próximo que tengo que reforzar: profundizar en autenticación y seguridad más allá de lo básico, y tomarme en serio las pruebas desde el principio de un proyecto y no como un requisito a cumplir al final. Si algo me deja este reto es ganas de seguir construyendo cosas así, cada vez con menos partes a medio entender y más partes que realmente domino.